

PRACTICAL WEEK-5

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-5

Task-1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define SIZE 5

int main()
{
    int pipefd[2];
    pid_t pid;
    int data[SIZE];
    int received[SIZE];

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        // Child process - Consumer
        close(pipefd[1]);

        printf("Consumer (Child) started. PID = %d\n", getpid());

        read(pipefd[0], received, sizeof(received));

        printf("Consumer received data: ");

        for (int i = 0; i < SIZE; i++)
        {
            printf("%d ", received[i]);
        }
    }
}
```

```

        close(pipefd[0]);
        exit(0);
    }
    else
    {
        // Parent process - Producer
        close(pipefd[0]);

        printf("Producer (Parent) started. PID = %d\n", getpid());

        for (int i = 0; i < SIZE; i++)
        {
            data[i] = (i + 1) * 10;
        }

        printf("Producer generated data: ");

        for (int i = 0; i < SIZE; i++)
        {
            printf("%d ", data[i]);
        }

        printf("\n");

        clock_t start = clock();

        write(pipefd[1], data, sizeof(data));

        clock_t end = clock();

        double time_taken =
            ((double)(end - start)) / CLOCKS_PER_SEC;

        printf("Communication time: %f seconds\n", time_taken);

        close(pipefd[1]);

        wait(NULL);

        printf("Producer completed.\n");
    }

    return 0;
}

```

```

saivhamsh@Varshini-PC: ~/OSSP_FIFO$ ./server
Server started.
Waiting for client messages...
Client Message: hello server
Response sent to client.
Client Message: how are you?
Response sent to client.
Client Message: exit
Server shutting down...
saivhamsh@Varshini-PC: ~/OSSP_FIFO$

```

```

saivhamsh@Varshini-PC: ~/OSSP_FIFO$ ./client
Client started.
Enter message: hello server
Server Response: Server processed: hello server
Enter message: how are you?
Server Response: Server processed: how are you?
Enter message: exit
saivhamsh@Varshini-PC: ~/OSSP_FIFO$

```

Observation: The program successfully implemented producer-consumer communication using an anonymous pipe. The parent process generated the data 10 20 30 40 50 and sent it through the pipe. The child process successfully received and displayed the same data. The communication time was measured as 0.000042 seconds. The parent used wait() to synchronize with the child process.

Task 2: Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    // First child: executes ls -l
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid1 == 0)
    {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls failed");
        exit(1);
    }

    // Second child: executes grep ".c"
    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork failed");
        exit(1);
    }
}
```

```

}

// Second child: executes grep ".c"
pid2 = fork();

if (pid2 < 0)
{
    perror("fork failed");
    exit(1);
}

if (pid2 == 0)
{
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep failed");
    exit(1);
}

// Parent closes both ends of the pipe
close(pipefd[0]);
close(pipefd[1]);

// Wait for both child processes
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;
}

```

```

saivihamsh@Varshini-PC: ~/OSSP_FIFO_Signal$ nano signals.c
saivihamsh@Varshini-PC:~/OSSP_FIFO_Signal$ gcc signals.c -o
signals
saivihamsh@Varshini-PC:~/OSSP_FIFO_Signal$ ./signals
Signal handling program started.
Process ID: 634
Waiting for signals...
Program is running...
Program is running...
Program is running...

SIGUSR1 received. User-defined signal handled.
Program is running...

saivihamsh@Varshini-PC:~/OSSP_FIFO_Signal$ kill -SIGUSR1 634
saivihamsh@Varshini-PC:~/OSSP_FIFO_Signal$

```

Observation: The program successfully implemented the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. The first child process executed `ls -l` and redirected its output to the pipe, while the second child process executed `grep ".c"` and received the input from the pipe. The program successfully displayed the `.c` file and completed the pipeline execution.

Result: The pipeline `ls -l | grep ".c"` was successfully implemented using system calls